MCMC algorithms

José G. Dias

Table of contents

1. Summary

This lecture discusses the workhorses of Bayesian estimation, the MCMC algorithms. We start from the Gibbs sampler that provides an intuitive understanding of the process underlying sampling estimation. We illustrate it using the estimation of the parameters of a bivariate normal distribution.

Often the likelihood and prior are not conjugate, then the posterior distribution is not given by a close-form solution. In Bayesian statistics, obtaining an analytical expression for the posterior distribution is often not feasible and numerical methods such as Markov Chain Monte Carlo (MCMC) algorithms or Variational Inference are employed to approximate the posterior distribution. Numerical methods are used to sample from the posterior distribution such as MCMC algorithms (e.g., Hamiltonian Monte Carlo implemented in Stan).

2. The Gibbs sampler

2.1 Definition

The Gibbs sampler decomposes the drawing from the joint posterior distribution into smaller problems of sampling from the conditional posterior distributions (that tend to be simpler).

For example, let us assume a model with parameters $\theta = (\theta_1, \theta_2)$, i.e., parameters vector θ can be broken into two blocks of parameters θ_1 and θ_2. In this case, the bivariate Gibbs sampler would be:

1. Define the setting of the MCMC algorithm (e.g., number of iterations, number of burn-in draws).
2. Choose a starting value for $\theta_2^{(1)}$. Set $r \leftarrow 2$.
3. Draws $\theta_1^{(r)}$ from $p\left(\theta_1 | \theta_2^{(r-1)}, y\right)$.
4. Draws $\theta_2^{(r)}$ from $p\left(\theta_2 | \theta_1^{(r)}, y\right)$.
5. $r \leftarrow r + 1$.
6. Repeat steps 3 to 5 for the desired number of iterations.
7. Post-processing the chains by producing descriptive statistics and plots.

2.2 Example for the bivariate normal distribution

This first example illustrates the Gibbs sampler with the bivariate normal distribution. We consider the bivariate normal posterior distribution such that

$$\theta = \begin{pmatrix} \theta_1 \\ \theta_2 \end{pmatrix} \sim N\left[\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}\right]$$

From the proprieties of the normal distribution, we can derive the conditional distributions:

$\theta_1 | \theta_2, y \sim N(\rho\theta_2, 1 - \rho^2)$.

Thus, the Gibbs sampler iterates between this two conditional distributions:

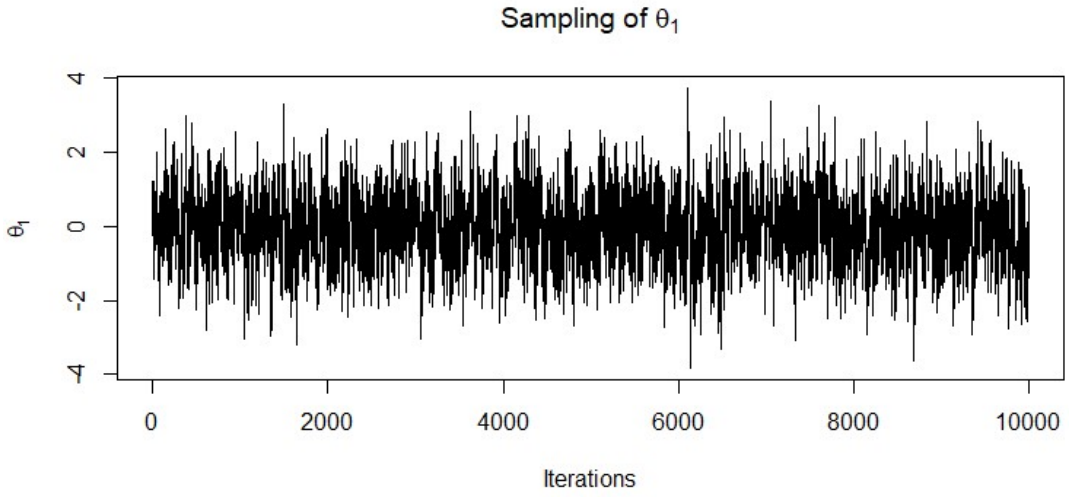
$$\theta_1 | \theta_2, y \sim \rho\theta_2 + \sqrt{1 - \rho^2} N(0,1)$$

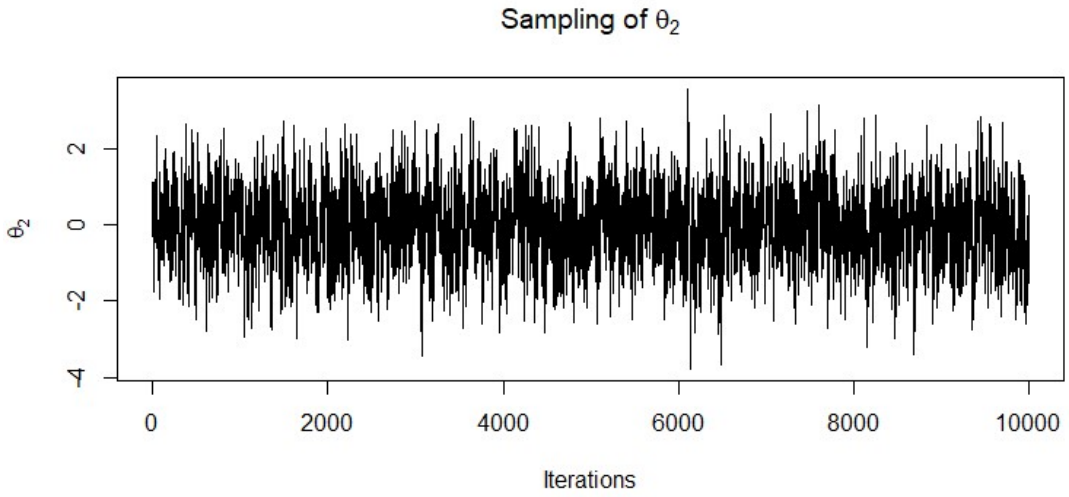
$$\theta_2 | \theta_1, y \sim \rho\theta_1 + \sqrt{1 - \rho^2} N(0,1)$$

Now we illustrate the estimation in the following context: 10000 total draws, 1000 burn-in draws, and ρ known and equal to 0.8. Set $\theta_1^{(1)} = \theta_2^{(1)} = 0$.

```
# Package for Greek characters

#install.packages('latex2exp')
library(latex2exp)

# Initialization

rho        <- 0.9
burnin     <- 1000
```

```r
n_iter     <- 10000
theta1     <- rep(0,times=n_iter)
theta2     <- rep(0,times=n_iter)

# Iterations

set.seed(123)

for (i in 2:n_iter){

  # Sampling theta1
  theta1[i] <-  rho*theta2[i-1]+sqrt(1-rho^2)*rnorm(1)

  # Sampling theta1
  theta2[i] <-  rho*theta1[i]+sqrt(1-rho^2)*rnorm(1)
}

# Concatenate both thetas

theta <-cbind(theta1,theta2)

par(mfrow = c(2, 1))
xx <- seq(1, n_iter)
plot(xx,theta[,1],type='l', ylab=TeX(r"($\theta_1$)"),
     xlab='Iterations', main = TeX(r"(Sampling of $\theta_1$)"))
plot(xx,theta[,2],type='l', ylab=TeX(r"($\theta_2$)"),
     xlab='Iterations', main = TeX(r"(Sampling of $\theta_2$)"))
```



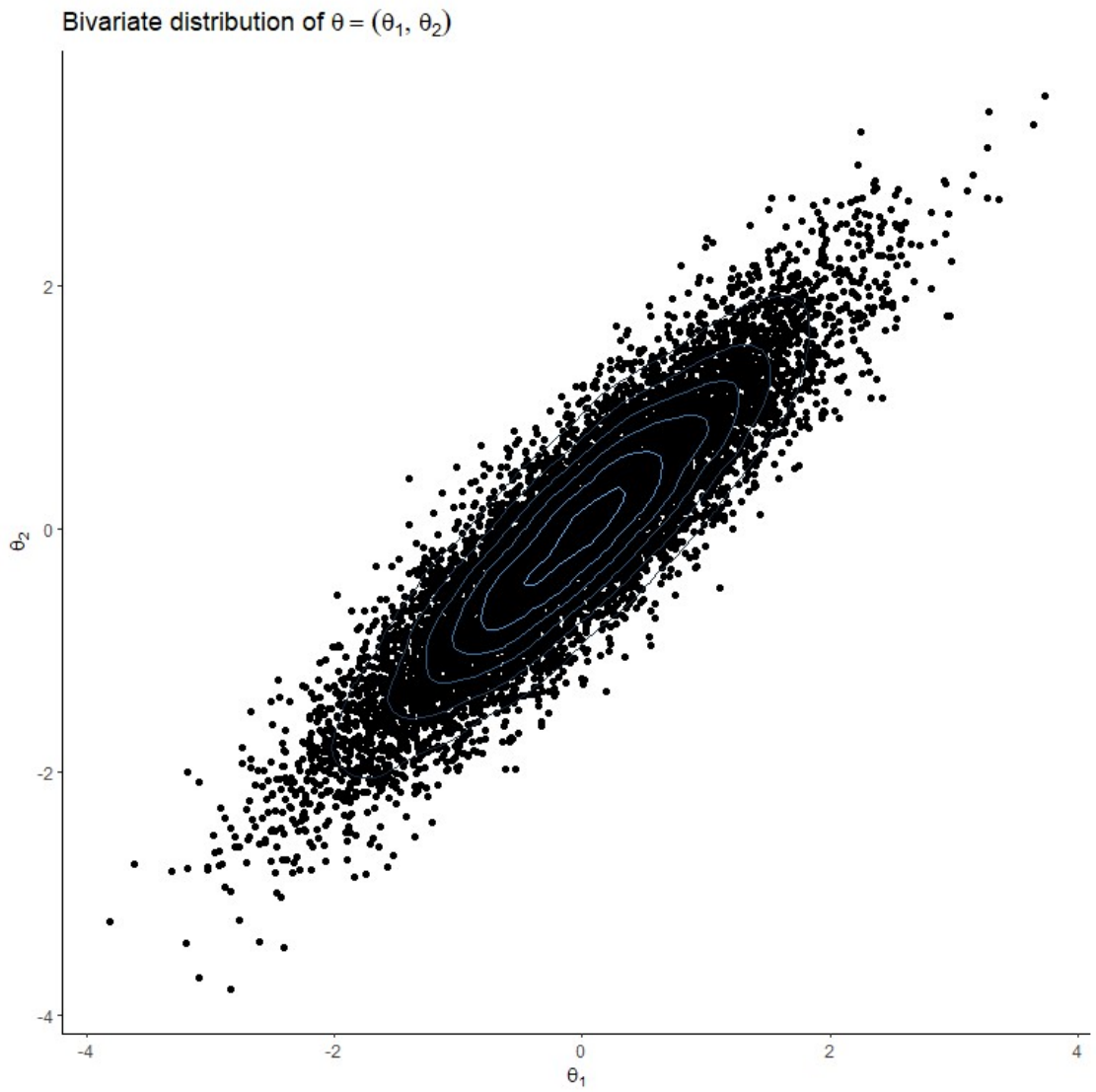



```
# Reset plotting layout

par(mfrow = c(1, 1))
```

The density plot, after removing the burn-in period, shows that the data set draws come from the bivariate normal distribution.

```
library(ggplot2)

# Remove the burn-in

data <- as.data.frame(theta[(burnin+1):n_iter,])

# 2D density plot

p <- ggplot(data, aes(x = data[,1], y = data[,2]))+
```

```
        geom_point() +
        stat_density2d(aes(colour = after_stat(level)))+
        ggtitle(TeX(r"(Bivariate distribution of $\theta =
(\theta_1,\theta_2))"))+
        xlab(TeX(r"($\theta_1$)"))+
        ylab(TeX(r"($\theta_2$)"))+
        theme_classic()+
        theme(legend.position = "none")
p
```



Finally, we can obtain the estimates of the mean, variance-covariance matrix, and correlation matrix.

```
# Print the mean and covariance samples

print("Sample means:")
```

```
[1] "Sample means:"
```

```
colMeans(theta[])
```

```
     theta1      theta2
-0.02398046 -0.02658384
```

```
print("Sample covariance matrix:")
```

```
[1] "Sample covariance matrix:"
```

```
var(theta)
```

```
          theta1    theta2
theta1 0.9532043 0.8562223
theta2 0.8562223 0.9580720
```

```
print("Sample correlation matrix:")
```

```
[1] "Sample correlation matrix:"
```

```
cor(theta)
```

```
          theta1    theta2
theta1 1.0000000 0.8959721
theta2 0.8959721 1.0000000
```

3. Metropolis-Hastings algorithm

3.1 The algorithm

The Metropolis-Hastings (HM) picks candidate draws from the specified candidate generating density and decides based on a acceptance probability if the draw is accepted or rejected. If not accepted, the new draw is the same as the previous one. This algorithm can be defined in the following steps:

1. Set up sampler specifications (e.g., number of iterations, number of burn-ins draws).
2. Choose a starting value for θ.
3. Draw θ^* from the candidate generating density.
4. Calculate the acceptance probability $\alpha\left(\theta^{(r-1)}, \theta^*\right)$.
5. Set $\theta^{(r)} = \theta^*$ with probability $\alpha\left(\theta^{(r-1)}, \theta^*\right)$ or else set $\theta^{(r)} = \theta^{(r-1)}$.
6. Repeat steps 3 to 5 for $r = 1, \ldots, R$.
7. Post-process the chain values (e.g., compute mean, variance).

The Gibbs sampler is a special case of the Metropolis-Hastings algorithm that takes advantage of the conditional distributions of the joint distribution. In that case, the candidate generating density is the conditional distribution and the probability of acceptance is 1.

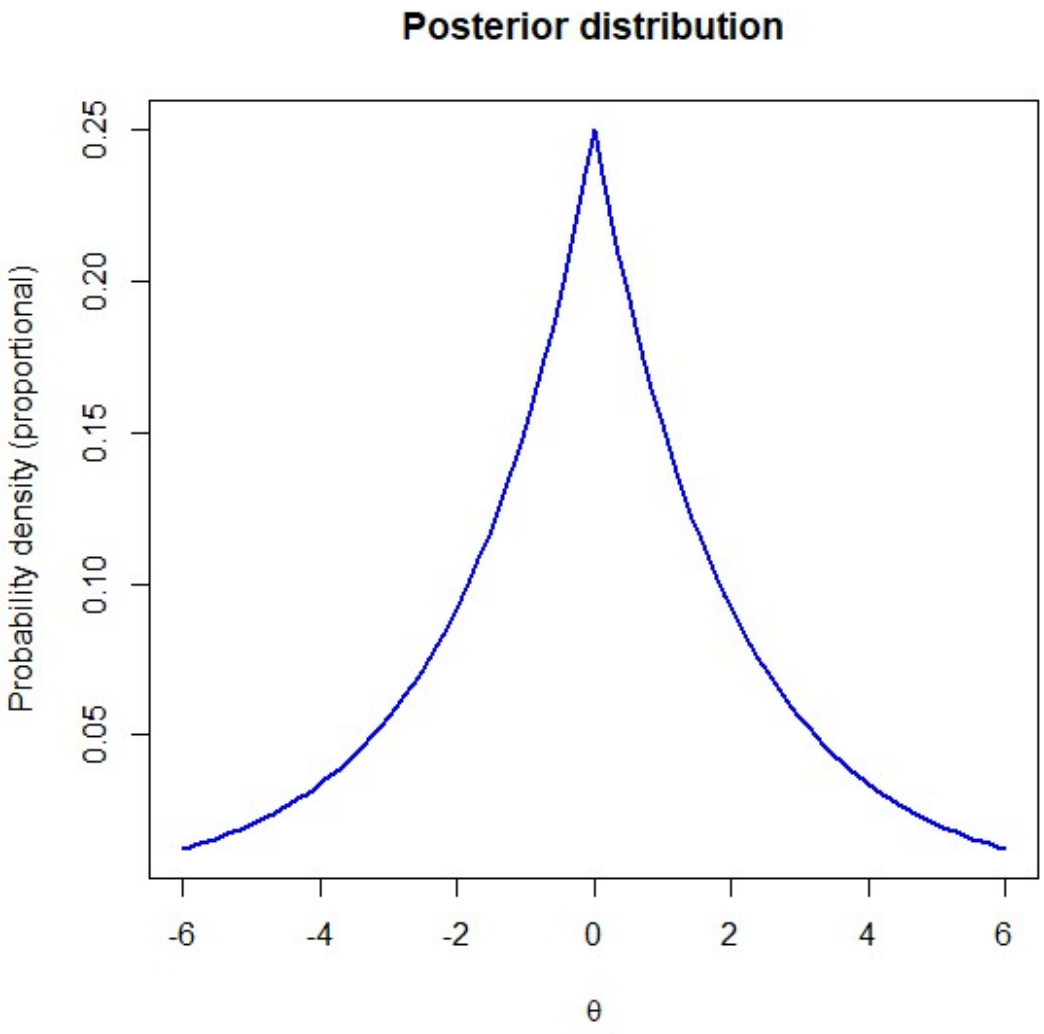
3.2 Example with a random walk chain

We illustrate the MH algorithm with the sampling from the posterior distribution given by

$$p(\theta|y) \propto \exp\left(-\frac{|\theta|}{2}\right),$$

which is proportional to a density, but not a density. To be a density, it has to be $\frac{1}{4}\exp\left(-\frac{|\theta|}{2}\right)$. Thus, the constant of proportionality is $1/4$.

```
# Define the function

thetafunction <- function(x) 0.25*exp(-0.5*abs(x))

# Plot the Gaussian distribution using the curve function

curve(thetafunction, from = -6, to = 6, col = "blue", lwd = 2,
      main = "Posterior distribution", xlab = TeX(r"($\theta$)"), ylab =
"Probability density (proportional)")
```



We have many methods to generate proposal candidates. The **random walk chain MH algorithm** draws from the process

$\theta^* = \theta^{(r-1)} + z,$

where z is a random normal variable that gives the increment, i.e.,

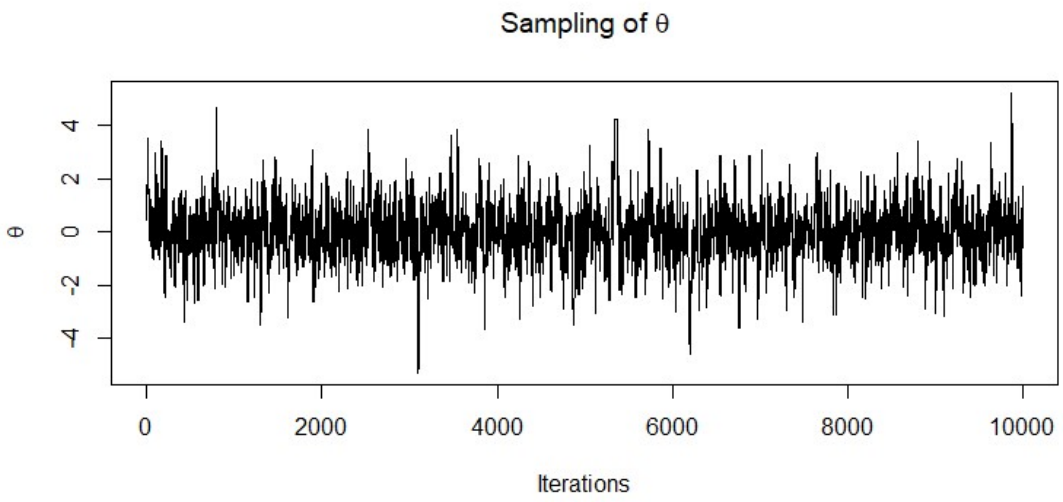
$\theta^* \sim N\left(\theta^{(r-1)}, c^2\right),$

where c is a tuning parameter. In this case, the acceptance probability is given by

$$\alpha\left(\theta^{(r-1)}, \theta^*\right) = \min\left[\exp\left(\frac{\left|\theta^{(r-1)}\right| - |\theta^*|}{2}\right), 1\right]$$

```
# Package for Greek characters

#install.packages('latex2exp')
```

```r
library(latex2exp)

# Initialization

set.seed(123)
burnin    <- 1000
n_iter    <- 10000
rw_c      <- 1    # sd of the normal
theta     <- rep(0,times=n_iter)
theta[1]  <- 1    # initialize at 1
counter   <- 0    # counter of the accepted draws
acc_prb   <- 0    # acceptance probability
theta_rw  <- 0    # candidate from RW

# Iterations

set.seed(123)

for (i in 2:n_iter){

  # Draw a candidate for random walk chain

  theta_rw <- theta[i-1] + rw_c*rnorm(1)

  # Calculate acceptance probability

  acc_prb <-  min(exp(0.5*(abs(theta[i-1]))-abs(theta_rw )),1)

  # Accept candidate draw with probability acc_prob

  if (acc_prb > runif(1,0,1)) {
    theta[i] <- theta_rw
    counter  <- counter+1}
  else{
    theta[i] <- theta[i-1]
    end}
}

# Plot of the chain

xx <- seq(1, n_iter)
plot(seq(1, n_iter),theta,type='l', ylab=TeX(r"($\theta$)"),
     xlab='Iterations', main = TeX(r"(Sampling of $\theta$)"))
```

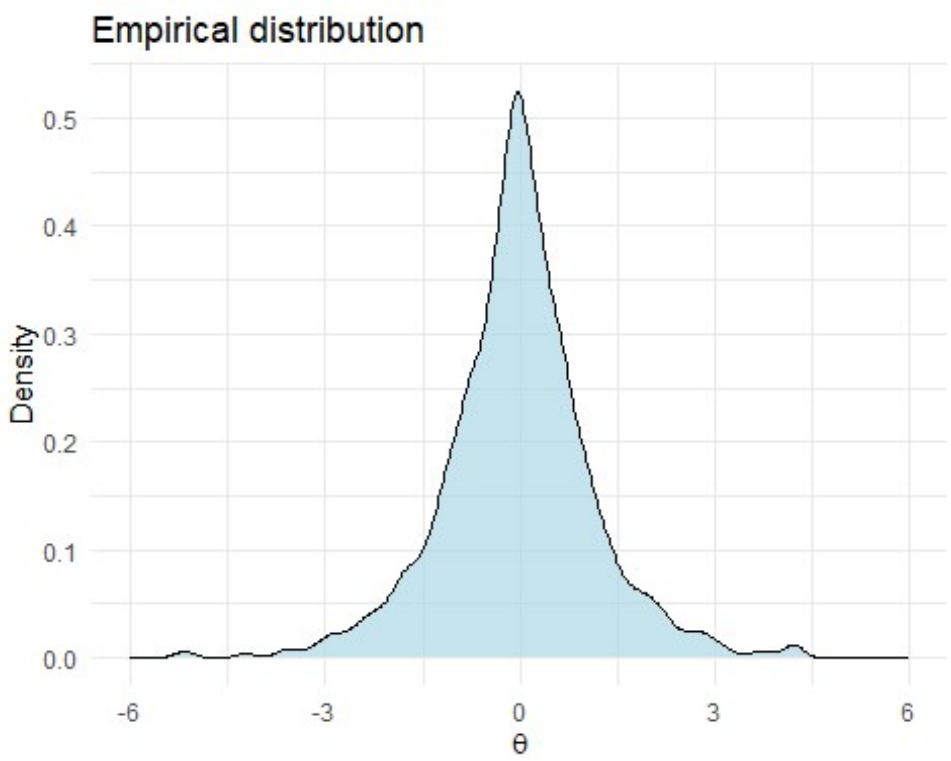


We can obtain the proportion of accepted draws (mixing), the estimates of the mean and variance after burn-in period.

```
# Remove the burn-in

data <- theta[(burnin+1):n_iter]

# The post processing

print(paste("Proportion of accepted candidate draws:", counter/n_iter))

[1] "Proportion of accepted candidate draws: 0.5574"

print(paste("Sample mean:                          ", mean(data)))

[1] "Sample mean:                         -0.0297006593173331"

print(paste("Sample variance:                      ", var(data)))

[1] "Sample variance:                     1.306136200579"
```

Figure below shows the density of the draws:

```
# Create a density plot of the draws

data <- as.data.frame(data)
p <- ggplot(data.frame(x = data), aes(x = data)) +
       geom_density(fill = "lightblue", color = "black", alpha = 0.7) +
       labs(title = "Empirical distribution",
       x = TeX(r"($\theta$)"), y = "Density") +
       xlim(-6, 6)+
       theme_minimal()
p
```



Try other values of c tuning to understand the most appropriate values to get a good mixing.

The independent chain draws proposals from

$\theta^* \sim N(0, c^2)$.

4. The Hamiltonian Monte Carlo (HMC) algorithm

The Hamiltonian Monte Carlo (HMC) algorithm is another MCMC method, but it differs significantly from both the Gibbs sampler and the Metropolis-Hastings algorithm. It is inspired in Hamiltonian dynamics in physics to propose and accept new samples. This "Hamiltonian" idea is the combination of the potential energy (related to the negative log-posterior probability of the distribution) and the kinetic energy of a system, which simulates the dynamics of a physical system, where a particle moves through a potential energy landscape. The HMC algorithm introduces auxiliary variables to represent the momentum of the particle, which helps explore the space more efficiently and make good proposals. The proposed sample is then accepted or rejected based on a Metropolis acceptance criterion. The Stan implements the HMC algorithm.